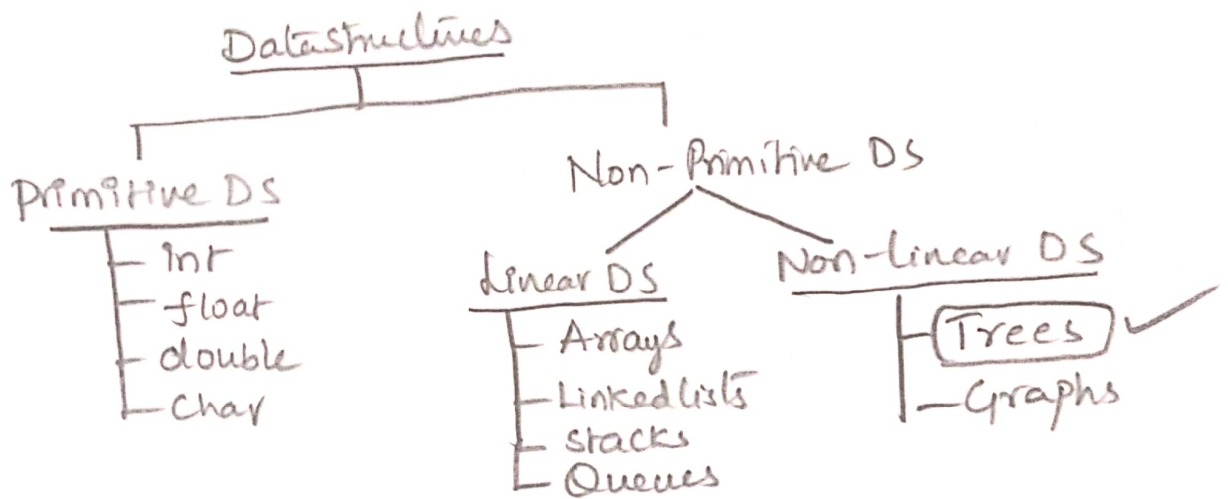




Tree :- It is non-linear Data Structure.

- It is the combination of Nodes and Edges.

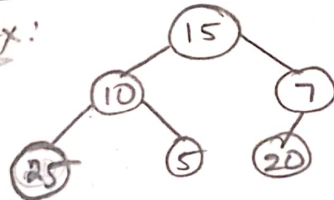
Node → It contains data (key) represented in circle ○

Edge → It is connecting link between any two nodes and represented with straight line.

Types of Trees : (Types of Binary Trees)

① Binary Trees → If any node contains at most (max) two child nodes

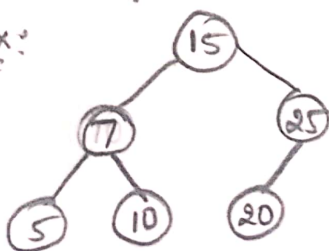
Ex:



② Binary Search Tree :-

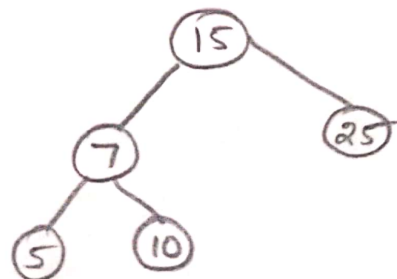
It is also Binary Tree such that left child less than parent
Right child greater than parent

Ex:

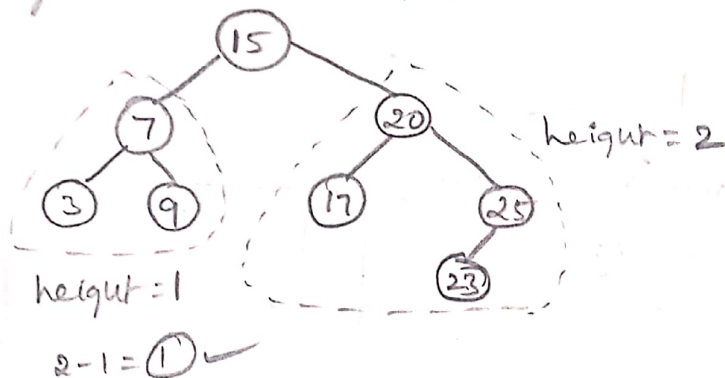


(strict)

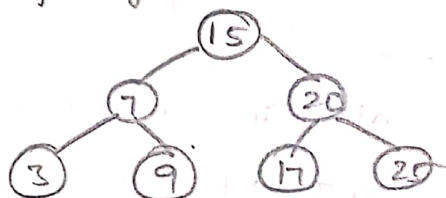
③ Full Binary Tree : Each node has either 0 OR 2 child nodes



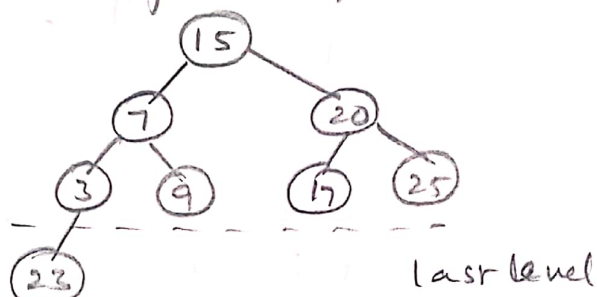
④ Balanced Binary Tree: The difference of heights of left and right subtrees is -1 or 0 or $+1$.



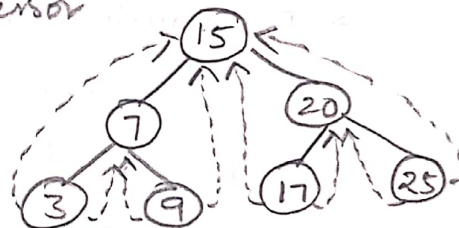
⑥ Perfect Binary Tree: All levels are full of nodes



⑤ Completed Binary Tree: All levels completely filled except the last filled left to right



⑦ Threaded Binary Trees: All left children with NULL refer to its inorder predecessor and right children with NULL refer to its inorder successor



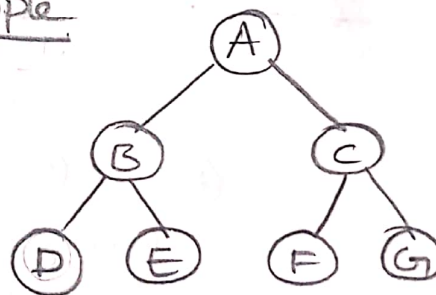
Tree Terminology (Concepts)

1) Root: - first node in tree (A)

2) Parent: A node which is predecessor of any node.
B' is parent of D and E

3) Child: A node which is descendant of any node
D, E are child of 'B'

Example



4) Siblings: The nodes which belong to same parent are called "Siblings". Here D and E are siblings belongs to same parent "B". (13)

5) Leaf: The nodes which ~~have~~ does not have child nodes are called "Leaf Nodes". D, E, F, G are child nodes.

6) Internal Node: The node which has atleast one node, is called "Internal Node" (Every Non-leaf Node is Internal node). A, B, C are Internal nodes

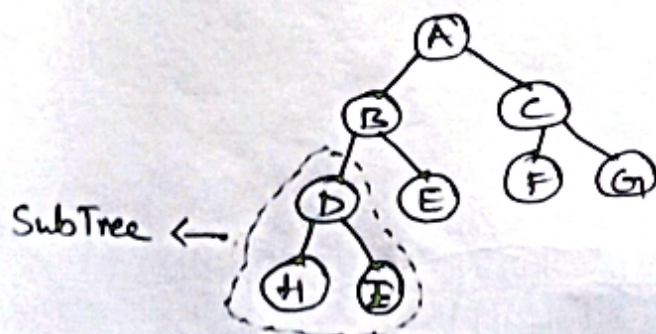
7) Degree: The total no. of children of a Node is called "Degree". Here Degree of B is "2".

8) Level: Each step from Top to bottom of a Tree is called "Level". Level starts with '0'. In the above example Tree, there are 3 levels (level 0, level 1, level 2)

9) Height: Total No. of edges from leaf node to a particular node is called "Height". Here from leaf node D to Root Node A is "2".

10) Depth: Total no. of edges from Root node to a particular node is called "Depth". Here from Root node A to leaf Node D is "2".

11) SubTree: It is part of the Tree as shown below:

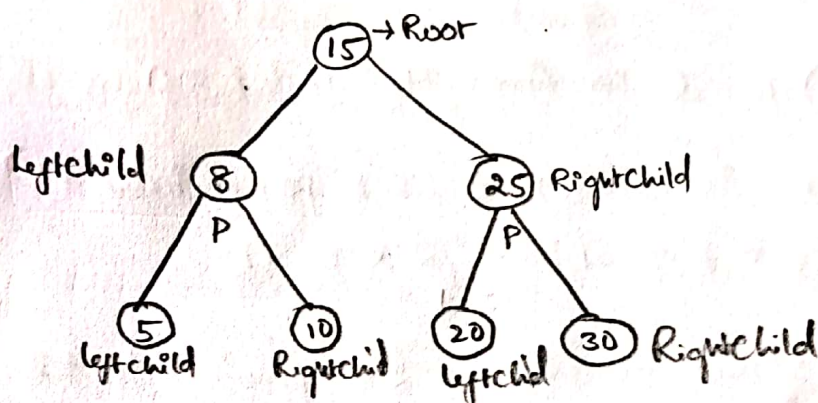


* Binary Sorted Tree (Ordered Binary Tree) * Binary Search Tree (BST) - Properties, Operations & Representation

→ It is also Binary Tree which satisfies the following characteristics

- Left child data should be less than its parent node data
- Right child Node data should be greater than its parent node
- There is no repeated value

Example:



→ Operations of Binary Search Tree:

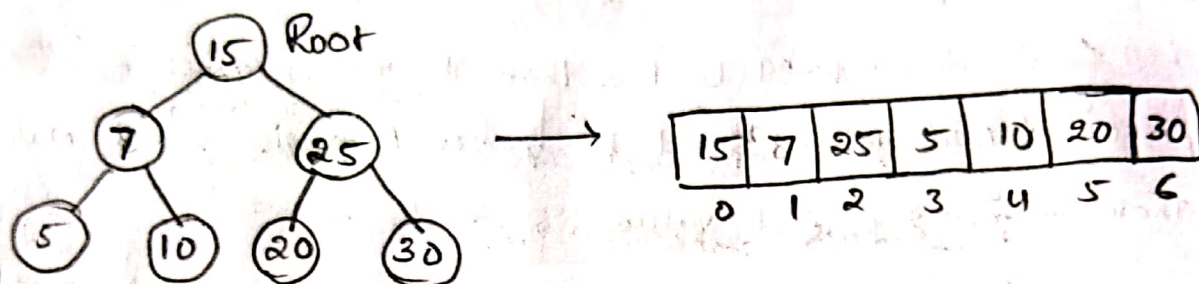
- ① Insert(key) → Inserts a new Node into Tree at leaf level
- ② delete (key) → deletes a node from tree
- ③ traverse() → displays all nodes in tree in different forms like inorder, preorder, postorder
- ④ find(key) / search(key) → Search a particular node in tree
- ⑤ max() → determines max node in tree
- ⑤ min() → determines Minimum Node in tree

The above specified details and operations are called
"ADT Binary Search Tree"

* Representation and Implementation

The Binary Search Tree is represented and implemented in 2 ways:

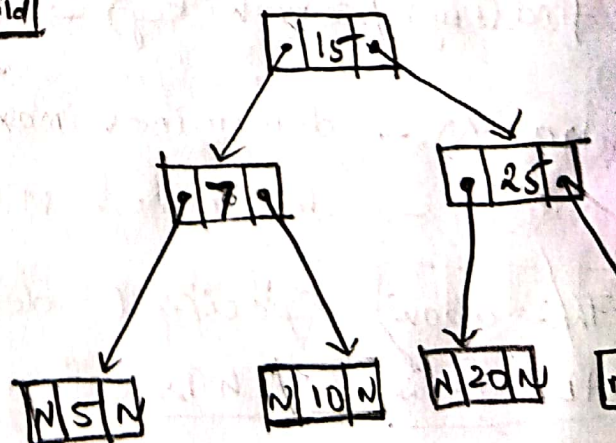
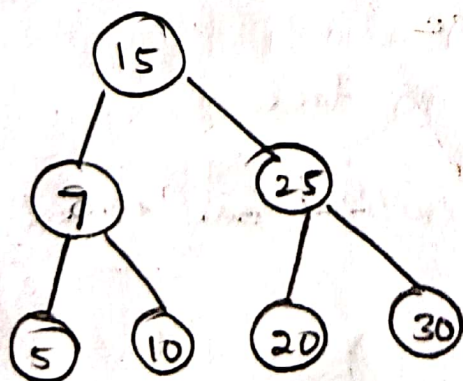
① By using Arrays:



- Root element (15) stored in Array at Index = 0
- Left child stored at Index = $(2 * \text{parent Index} + 1)$
 Ex: 7 stored at index = $2 * 0 + 1 = 1$
- Right child stored at Index = $2 * \text{parent Index} + 2$
 Ex: 25 stored at index = $2 * 0 + 2 = 2$

② By using Linked Lists:

Binary Trees or Binary Search Trees are implemented by using Double Linked lists because every node contains three parts as shown below:

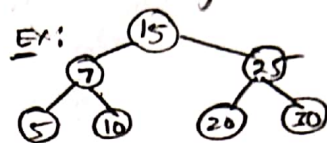


Construction of Binary Search Tree (BST) [Binary Search Tree] Page 6

Inserting Nodes into Binary Search Tree (BST)

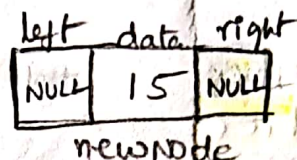
Binary search Tree $\rightarrow$ is also Binary Tree which satisfies the following

- a) left child should less than parent
- b) Right child should greater than parent



To Construct Binary Search Tree we use `insert()` operation as follows
`insert(root, key)` $\rightarrow$ inserts newNode into Binary Search Tree.

step 1: Create newNode and insert Data



step 2: if `(root == NULL)` /* Tree Empty */
return newNode;

step 3: if `(data < root->data)` /* less than parent Node */

root $\rightarrow$ left = `insert(root  $\rightarrow$  left, data)`; /* insert as left child */

else

root $\rightarrow$ right = `insert(root  $\rightarrow$  right, data)`; /* insert as Right child */

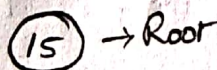
step 4: return root;

Example and Diagram:

Construct Binary Search Tree with given data 15, 7, 25, 20, 30, 10, 5

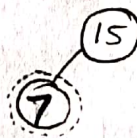
insert (root, 15)

Since Tree is Empty,
15 become Root Node



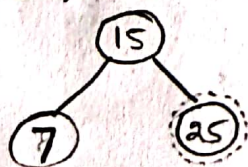
insert (root, 7)

Since $7 < 15$, insert as left child



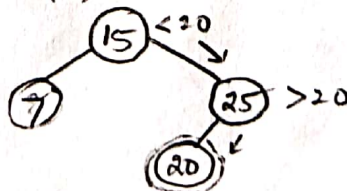
insert (root, 25)

Since $25 > 15$, insert as Right child



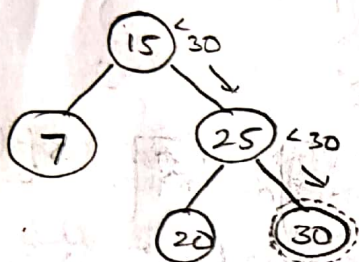
insert (root, 20)

$\rightarrow$ Since $20 > 15$, move to right tree and Compare with 25, since $20 < 25$. insert as left child



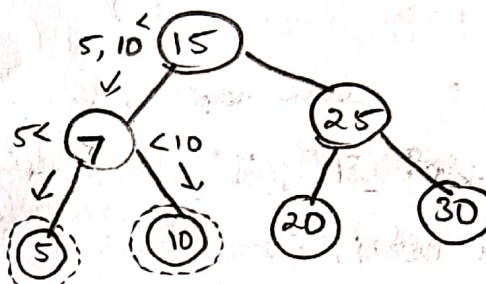
Insert(root, 30)

- Since $30 > 15$, move to right
- Compare with 25,
- Since $30 > 25$ insert as Right child



Insert(root, 10) and Insert(root, 5)

- Since $10, 5 < 15$ move to left
- Since $10 > 7$, insert as right child
- Since $5 < 7$, insert as left child



* Traverse() of Binary Search Tree (DFS in Trees)

* Tree Traversal Techniques (PreOrder, InOrder, PostOrder)

→ Traversing is the process of visiting & displaying all nodes in the Tree

→ There are three types of Traversing techniques

① PreOrder (Root, Left, Right)

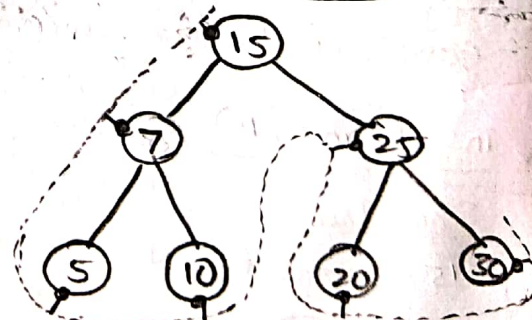
② InOrder (Left, Root, Right)

③ PostOrder (Left, Right, Root)

① PreOrder : → In this, we traverse by following Root, Left, Right

Algorithm: void preorder(struct Node *root)

Example Tree



Step 1: printf("%d", root->data);

Step 2: call preorder(root->left);

Step 3: call preorder(root->right);

preorder of above Tree
as follows

15 - 7 - 5 - 10 - 25 - 20 - 30

② InOrder :- In this, we traverse by following Left, Root, Right Page 8

Algorithm inorder(struct Node *root)

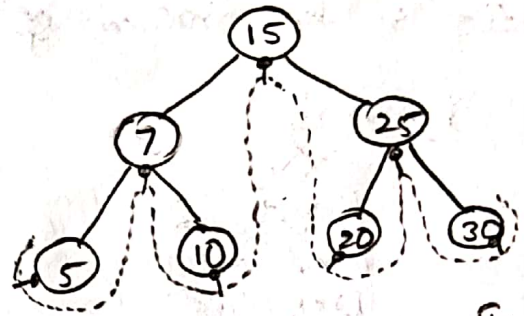
if (root == NULL)

step 1: Call inorder(root → left);

step 2: printf("%d", root → data);

step 3: Call inorder(root → right);

Example Tree



In order of above Tree
Is 5-7-10-15-20-25-30

③ Postorder → In this, we traverse by following Left, Right, Root

Algorithm

void postorder(struct Node *root)

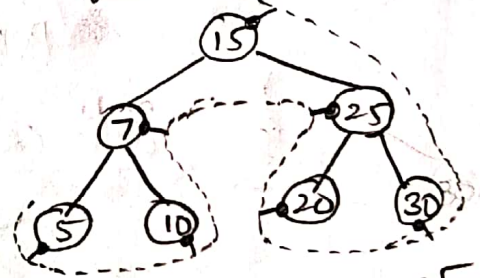
if (root == NULL)

step 1: Call postorder(root → left);

step 2: Call postorder(root → right);

step 3: printf("%d", root → data);

Example Tree



Post order of above Tree
Is 5-10-7-20-30-25-15

* delete (int key) operation in Binary Search Tree Page 9

- It is the process of delete a node in Tree

Algorithm:

Step 1: Read deleting node data (key)

Step 2: Find deleting node and its parent node

Step 3: Deleting node may be in one of the following cases

Case (i): Deleting Node with NO child (Leaf Node)

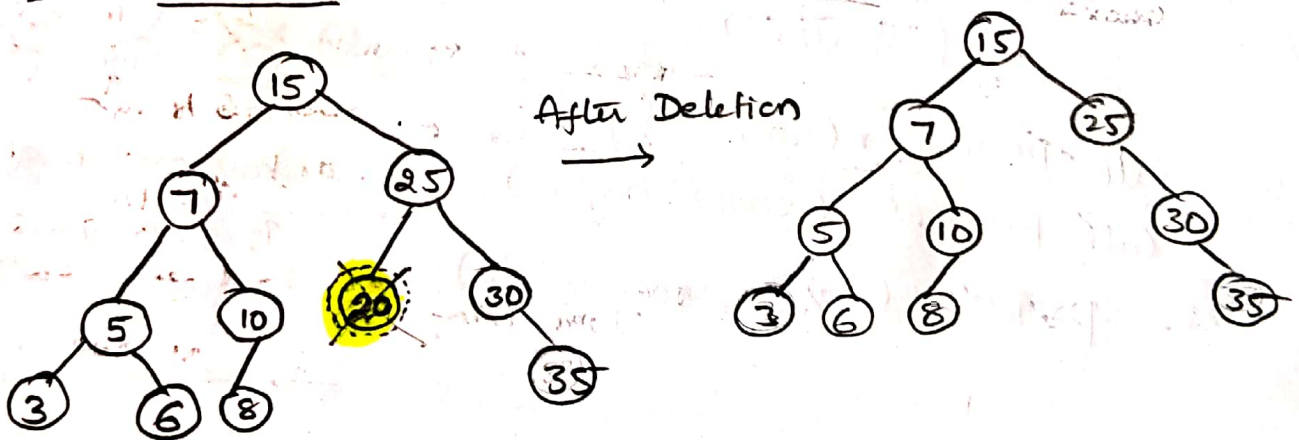
a) if deleting is left child,

parent $\rightarrow$ left = NULL

b) if deleting node is right child,

parent $\rightarrow$ right = NULL

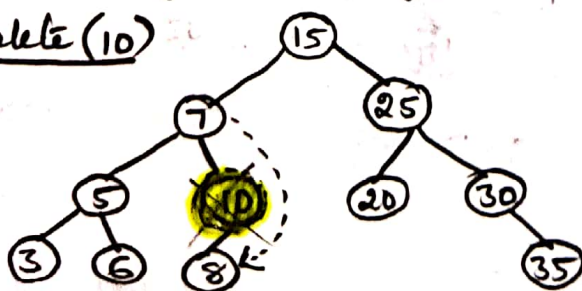
Ex: delete(20)



Case (ii) Deleting Node with one child

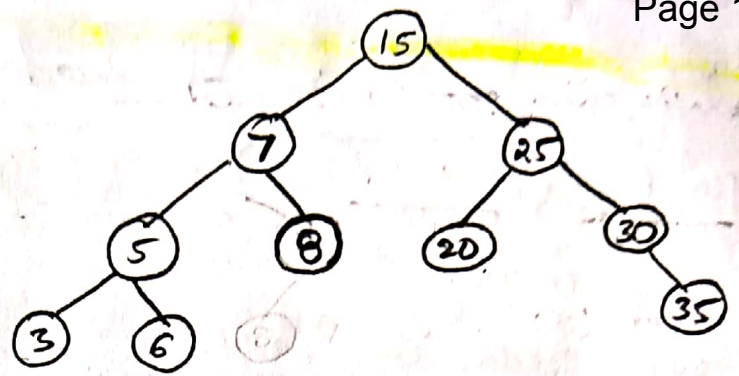
$\rightarrow$ provide link between parent of deleting node and child of deleting node

Ex: delete(10)



Here 10 $\rightarrow$ Deleting Node
7 $\rightarrow$ parent of 10
8 $\rightarrow$ child of 10

After Deleting 10, the tree will be



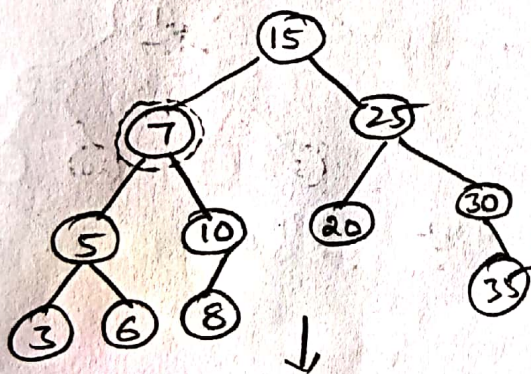
Case(iii): Deleting Node with two child

Step 1: Find Minimum Node in Right Subtree of deleting node
(OR)
Find Maximum Node in Left Subtree of deleting node

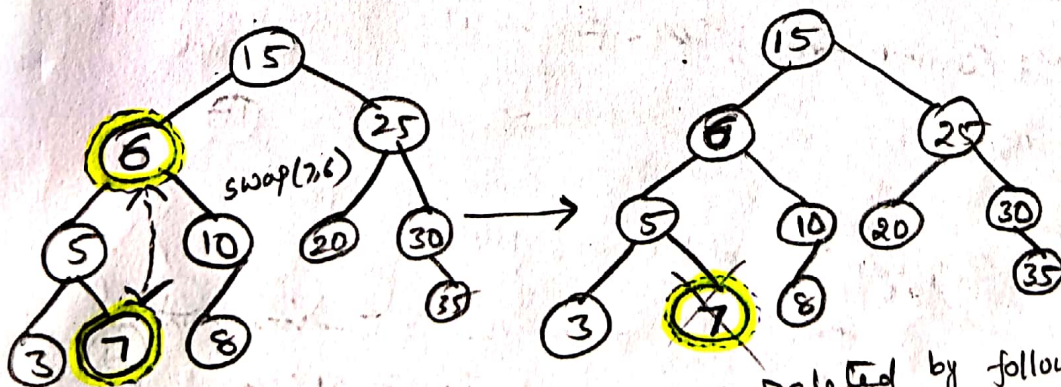
Step 2: Swap deleting node and Node found in previous Step-1

Step 3: The deletion comes to either case(i) or case(ii)
Now perform related delete case operation.

Example: delete(7)



- Here 7 is with two child, then
- Consider Maximum 6 in left Subtree
(OR)
Minimum 8 in Right Subtree
- Then swap (7, 6) OR swap(7, 8)
- Then Tree becomes as shown below



Now Deleting becomes
under Case(i)
i.e. Deleting Leaf Node

7 Deleted by following case(i) process

① find(Key) / search(Key) → It is process of finding a particular node in Tree

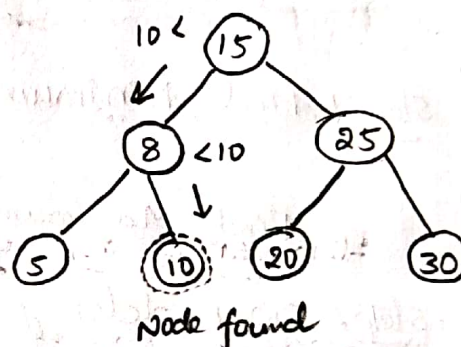
Step 1: Declare $*Current = root$

Step 2: Repeat step 2 until $Current \rightarrow data \neq Key$

Step 2: If $Key < Current \rightarrow data$
 $Current = Current \rightarrow left$
 else
 $Current = Current \rightarrow right$

Step 3: If $(Current == NULL)$
 print "Node not found"
 else
 print "Node found"

Example:
find(10)



② min() → It returns minimum node value in Tree.

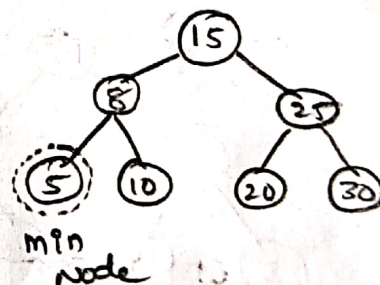
Step 1: Declare $*Current = root$, $*last$

Step 2: $last = Current$

Step 3: $Current = Current \rightarrow left$

Repeat step 2, step 3 until $Current \neq NULL$

Example



③ max() → It returns maximum node value in Tree.

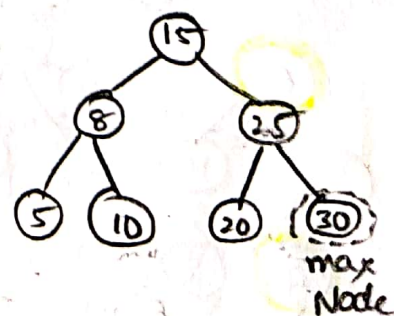
Step 1: Declare $*Current = root$, $*last$

Step 2: $last = Current$

Step 3: $Current = Current \rightarrow right$

Repeat step 2, step 3 until $Current \neq NULL$

Example

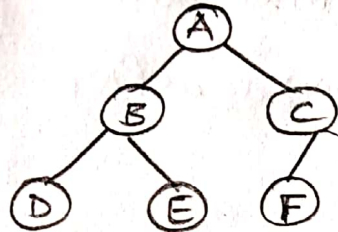


* Binary Trees and Construction of Binary Trees

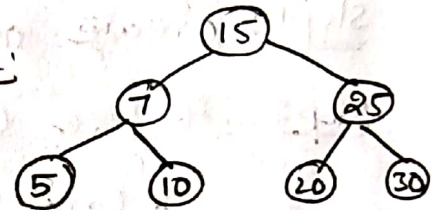
→ Binary Tree :- It is a non-linear data structure.

- In this, each node have at most (max) two child nodes. (left child and right child)

Example 1:



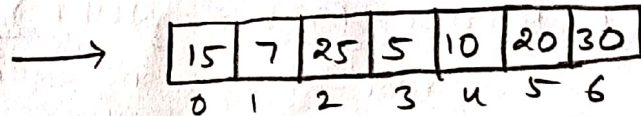
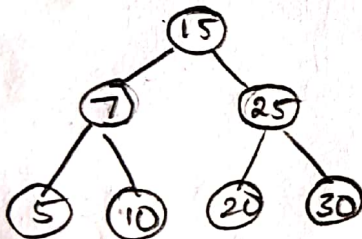
Example 2:



→ Representation and Implementation of Binary Tree

The Binary Tree is represented and implemented in 2 ways:

① By using Arrays:



→ Root element (15) stored at index = 0

→ Left child store at index = $(2 * \text{parent index} + 1)$

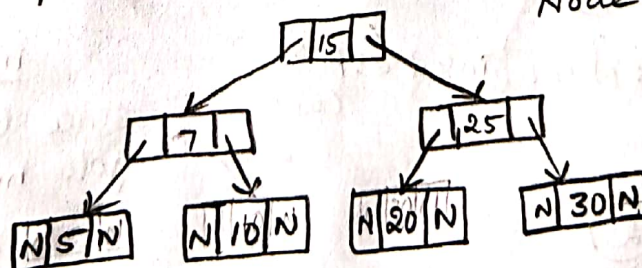
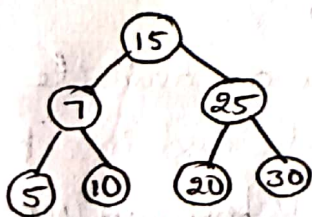
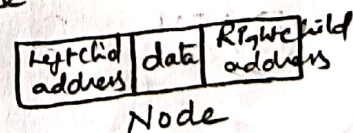
Ex: 7 store at index = $2 * 0 + 1 = 1$

→ Right child store at index = $(2 * \text{parent index} + 2)$

Ex: 25 store at index = $2 * 0 + 2 = 2$

② using Linked Lists:

Binary trees implemented using Double linked lists because every node contains three parts as shown

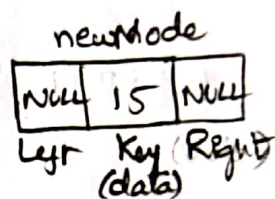


→ Construction of Binary Trees / Inserting Nodes into Binary Tree Page 13

To Construct Binary tree we use insert() operation as follows:

insert(root, Key) → inserts a newnode into Binary Tree

Step 1: create newNode and insert data



Step 2: If Tree Empty, make newNode as Root

if (root == NULL)

return newNode

Step 3: If Leftchild is NULL, insert newNode as leftchild

if (root → left == NULL)

root → left = newNode;

else If Rightchild is NULL, insert newNode as Rightchild

if (root → right == NULL)

root → right = newNode;

Otherwise, goto next level and insert as

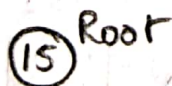
Repeat steps ①, ②, ③ Recursively in left and right Subtrees

Example and Diagram

Construct Binary tree for given elements 15, 7, 25, 5, 10, 20, 30

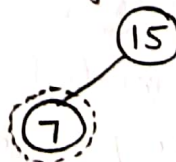
insert(15)

Since tree Empty, make it as Root



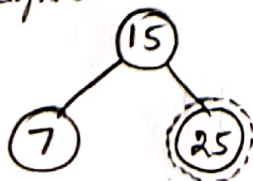
insert(7)

→ Since leftchild is null, insert as left



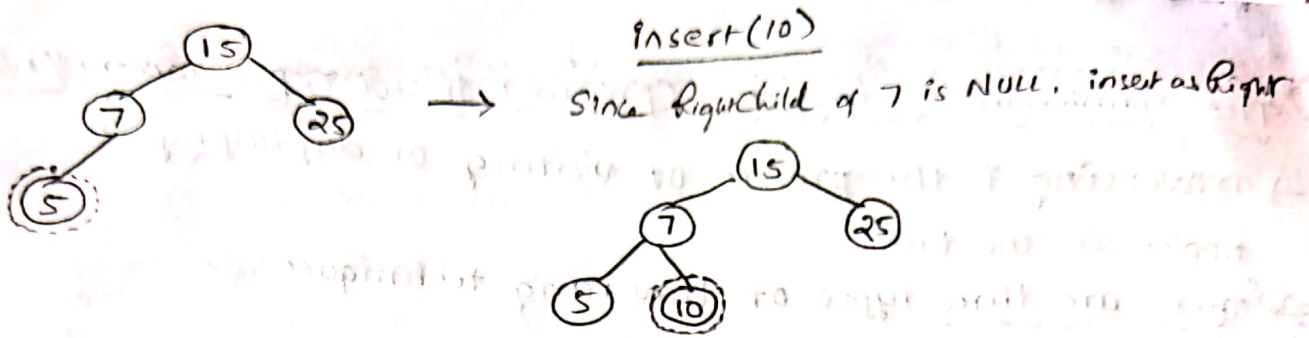
insert(25)

Since Rightchild is NULL, insert as Right child

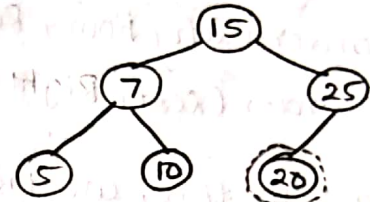


insert(5)

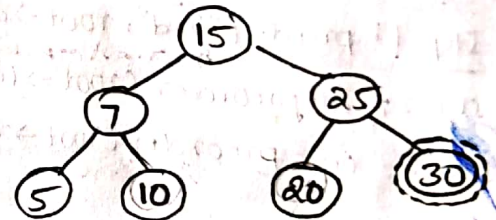
→ Since left, Right are available goto next level and insert as leftchild in left Subtree



- Insert(20)
- Since Left Subtree is Completed Repeat same process to form Right SubTree..
 - So insert 20 as left child in Right SubTree



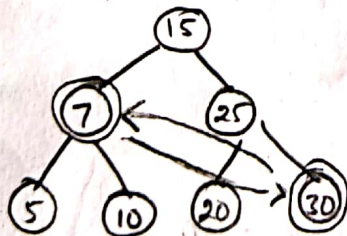
- Insert(30)
- Right child is NULL in right SubTree, So insert as Rightchild



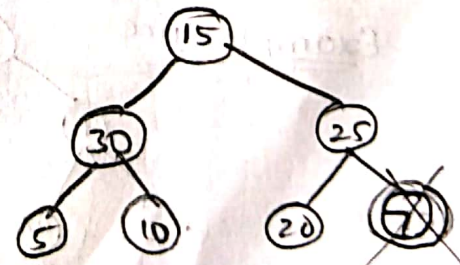
* Deleting a Node from Binary Tree:

- Step 1: Find the Deleted Node in the Tree
- Step 2: Find the Deepest Node (last level right end Node)
- Step 3: Swap Deleted Node and Deepest node
- Step 4: Delete the Node

Example: delete(7)



Here 7 - Deleted Node
30 - Deepest Node
Swap(7, 30)
Delete 7



* (Note: same as Binary Search Tree)

Tree Traversal Techniques:- (preorder, inorder, post order) Page 15 of Binary Tree

→ Traversing is the process of visiting or displaying all nodes in the tree.

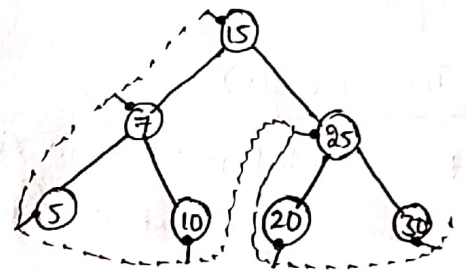
→ There are three types of traversing techniques

- ① preorder (Root, left, Right)
- ② inorder (left, Root, Right)
- ③ post order (left, Right, Root)

} also called **DFS** in Trees

① preorder: In this, we traverse by following Root, left, Right

Example tree



preorder of above tree
as follows

15-7-5-10-25-20-30

② Inorder: In this, we traverse by following left, Root, Right

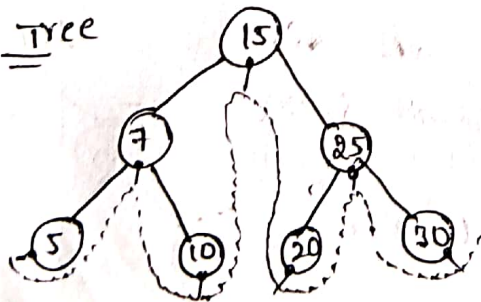
Algorithm inorder (struct Node *root)

Step 1: call inorder (root → left);

Step 2: printf ("%d", root → data);

Step 3: call inorder (root → right);

Example tree



in order of above tree is 5-7-10-15-20-25-30

⑧ post order :- In this, we traverse by following left, right, Page 16

Root

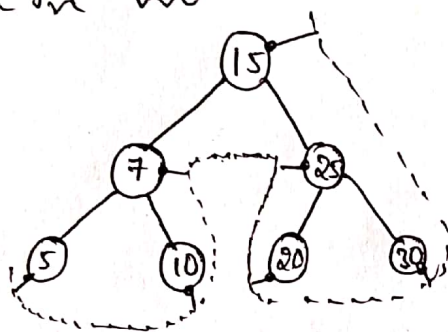
Algorithm postorder(struct Node root)

Step 1 :- call postorder(root → left);

Step 2 :- call postorder(root → right);

Step 3 :- printf("%d", root → data);

Example Tree

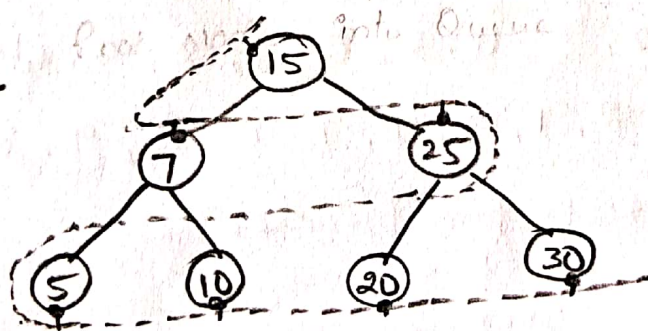


postorder of above tree is 5-10-7-25-20-30-15.

(BFS in trees)
* Level order Traversal of Binary Tree / Binary Search Tree

- It is special type of traversal, in this we visit the nodes level by level from top to Bottom and Left to Right in each
- This process uses Queue Data structure for FIFO order

Example



As shown above, we traverse and visit nodes from the root node to bottom level nodes and left to right. The result is as follows

15 - 7 - 25 - 5 - 10 - 20 - 30

from given Traversing orders

→ we can construct the Binary Trees (or) Binary Search Trees from given traversing orders also.

→ for this, we require minimum two traversal combinations to construct tree. They are

- ① Inorder and Preorder
- ② Inorder and Postorder

① Constructing from Inorder and Preorder:

Step 1: Read start node from Preorder and mark as Root (1st start is Root in in)

Step 2: Identify position of that node in Inorder Traversal and let it be X

Step 3: Form left subtree with nodes preceding (before) X and form right subtree with nodes succeed (After) X

Apply Recursively step ① to step ③ for all nodes in Preorder

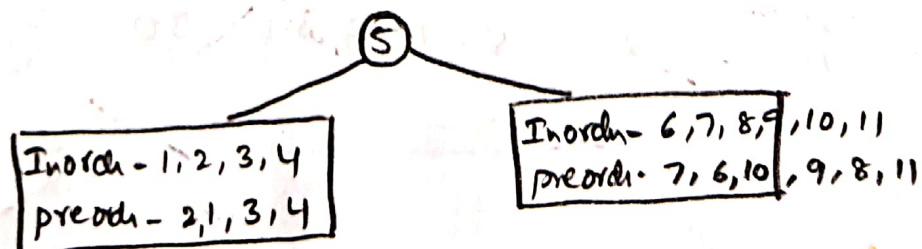
Example: Inorder: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
Preorder: 5, 2, 1, 3, 4, 7, 6, 10, 9, 8, 11

- Read ⑤ and make as Root from preorder

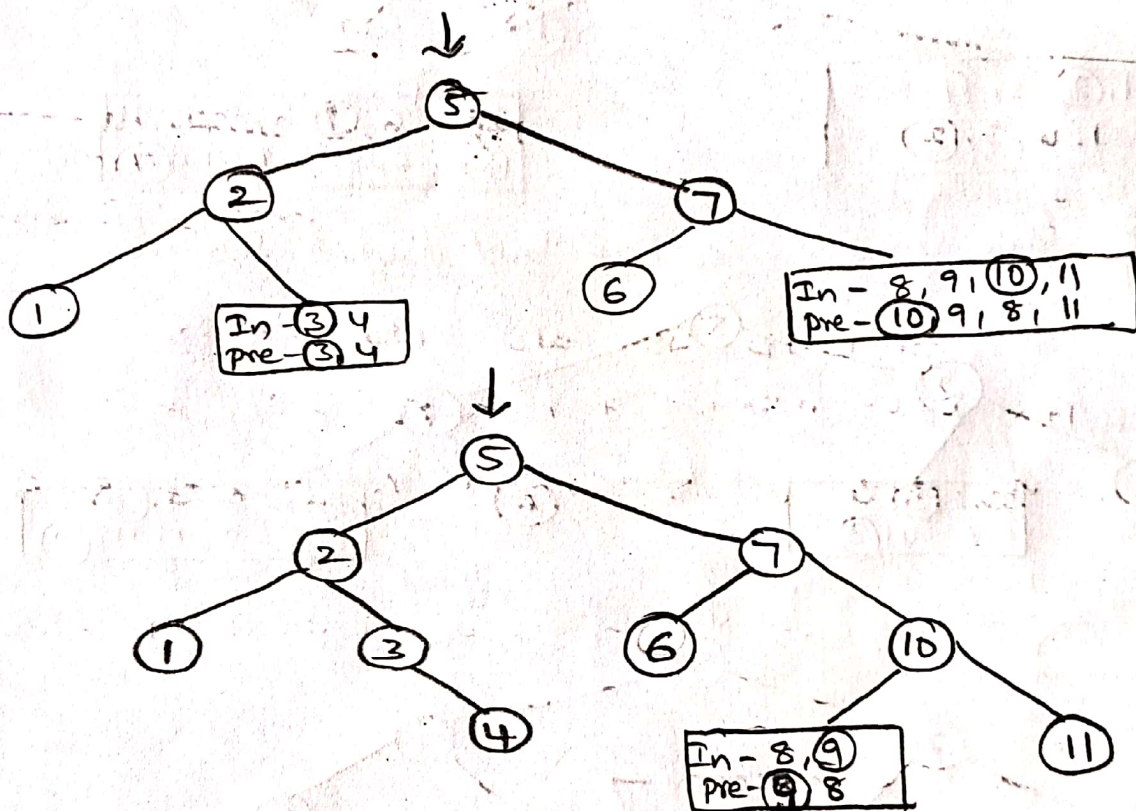
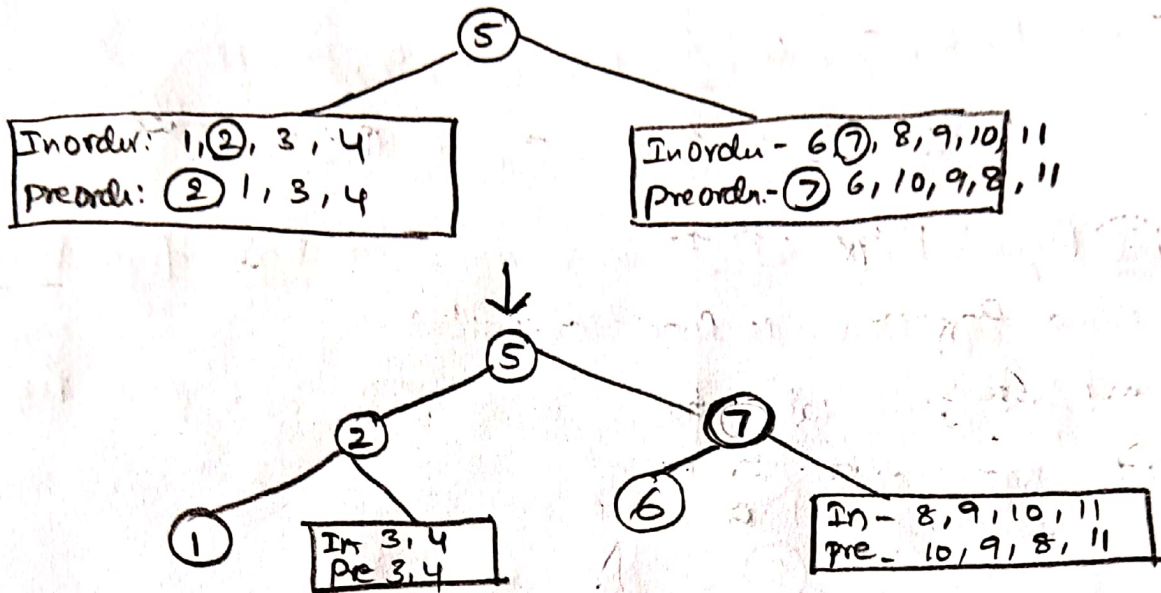
⑤

- Identify ⑤ in inorder - 1, 2, 3, 4, ⑤, 6, 7, 8, 9, 10, 11

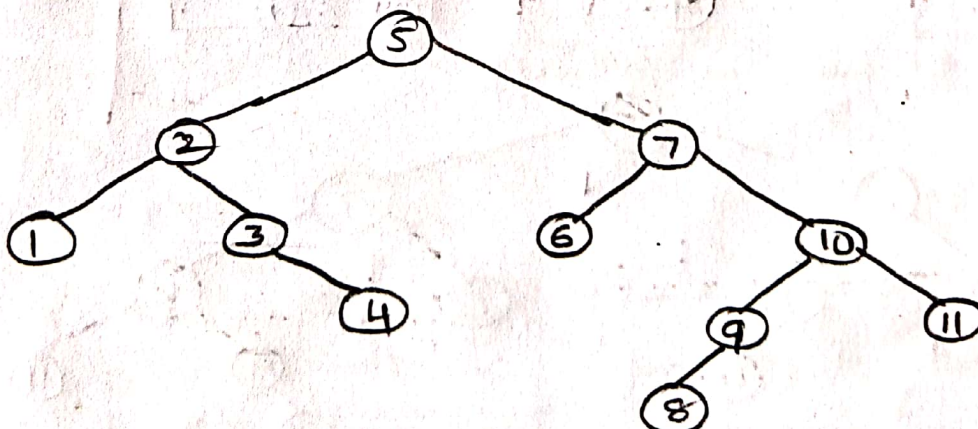
- Form Left Subtree with nodes before ⑤ and Right Tree after ⑤ as shown below



Repeat the same process for two subtrees



Finally we get Tree as follows



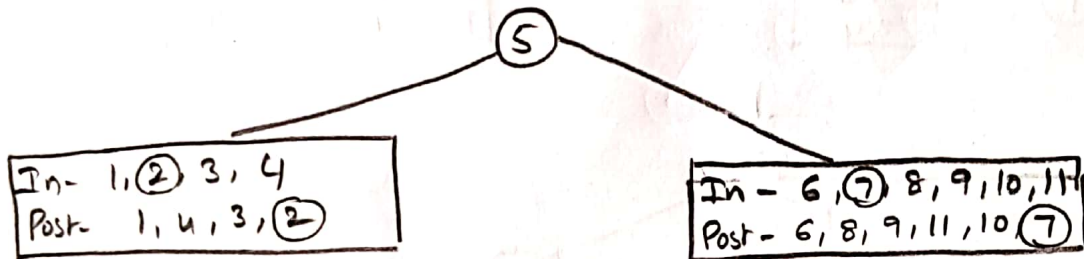
② Constructing from Inorder and Postorder

Inorder - 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11

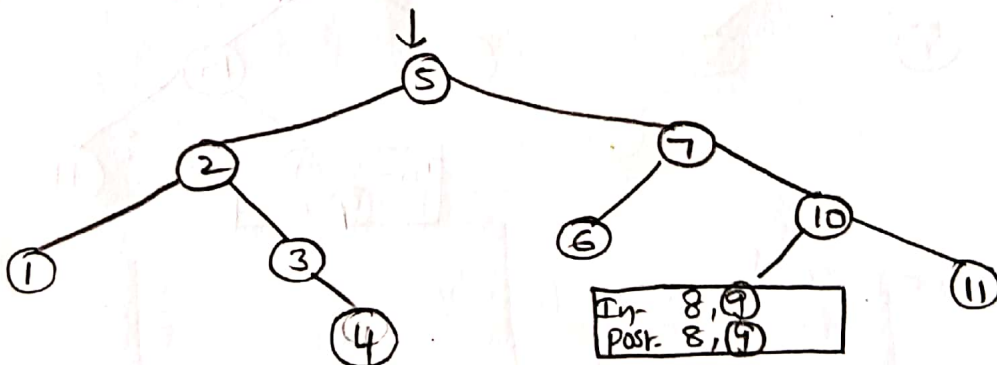
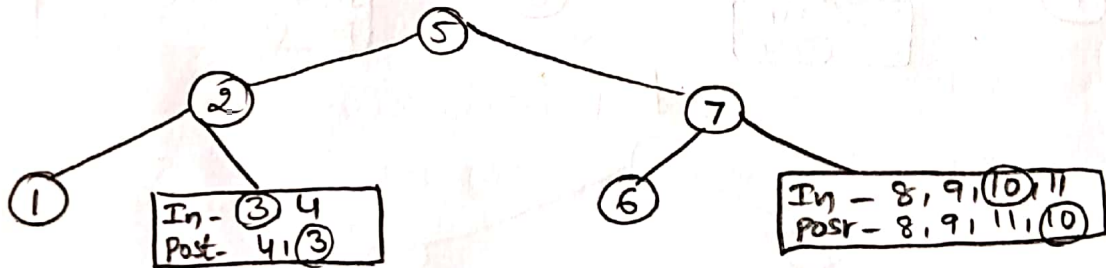
Postorder - 1, 4, 3, 2, 6, 8, 9, 11, 10, 7, 5

Step 1: Read Rightmost Node (5) and make as Root from Postorder
because Rightmost is Root in Postorder

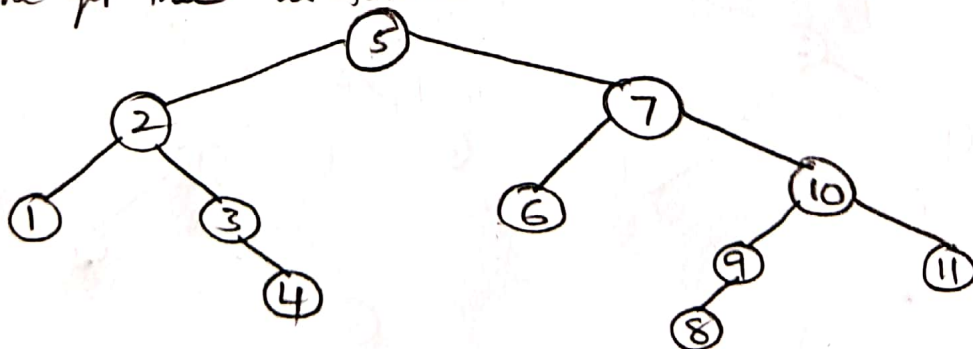
Step 2: Form Left Tree with nodes before (5) and form Right Tree
with nodes after (5)



Repeat same process for Left and Right Subtrees



finally we get Tree as follows:



Unit-4 Questions

- Imp
- ① Binary Search Tree
- Representation and Implementation
 - *Constructing BST
 - *Traversing
 - Deletion
- ② Binary Trees
- Representation and Implementation
 - *Constructing Binary Tree
 - *Traversing
 - Deletion

- ~~Notes~~
- ③ Level order Traversal
- ④ Constructing Binary Tree / Binary Search Tree from Traversing
 (Inorder + Postorder ^(ok) and Preorder) ~~Q~~
 Inorder + Preorder)
- ⑤ Types of Trees